

Bases de Dados Relacionais e SQL I

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercícios

Exercício 0: Configuração do Ambiente

Antes de começar, certifique-se de que tem o SQLite instalado no seu sistema.

Opção A: Instalação Manual

Se estiver a utilizar um sistema operativo baseado em Debian (como o Ubuntu), pode instalá-lo utilizando o apt:

1. Atualize a lista de pacotes:

```
sudo apt update
```

2. Instale o SQLite3:

```
sudo apt install sqlite3
```

3. Verifique a instalação:

```
sqlite3 --version
```

Opção B: Docker (Recomendado)

Se tiver o Docker instalado, pode utilizar o ambiente pré-configurado na pasta 02_support: 1. Navegue até 02_support. 2. Execute `docker compose up -d`. 3. Aceda ao contentor Python: `docker compose exec python-lab bash`.

Exercício 1: SQLite com Python

Nesta aula, utilizaremos o módulo nativo do Python `sqlite3` para interagir com uma base de dados SQLite.

1. Crie um novo ficheiro Python chamado `university_db.py`.
2. Importe o módulo `sqlite3` e estabeleça uma ligação:

```
import sqlite3
```

```
# Ligar (ou criar) o ficheiro da base de dados
conn = sqlite3.connect('university.db')
cursor = conn.cursor()
```

```
# O seu código será inserido aqui
```

```
conn.close()
```

Exercício 2: Definir o Esquema Universitário (DDL)

Utilizando `cursor.execute()`, crie as seguintes tabelas baseadas no cenário universitário:

1. University (id, name)
2. Department (id, name, univ_id)
3. Teacher (id, name, dept_id)
4. Course (id, name, dept_id, teacher_id)
5. Student (id, name)
6. Enrollment (stud_id, course_id)

Exemplo:

```
cursor.execute('''
CREATE TABLE IF NOT EXISTS University (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL
)
''')
conn.commit()
```

Exercício 3: Inserir Dados (DML)

Insira dados de teste nas suas tabelas. Tente utilizar consultas parametrizadas para prevenir SQL injection.

1. Insira 1 University.
2. Insira 2 Departments.
3. Insira 3 Teachers.
4. Insira 4 Courses.
5. Insira 5 Students e inscreva-os em várias cadeiras.

Exemplo:

```
teachers = [('Dr. Smith', 1), ('Prof. Jones', 1), ('Dr. Taylor', 2)]
cursor.executemany('INSERT INTO Teacher (name, dept_id) VALUES (?, ?)', teachers)
conn.commit()
```

Exercício 4: Consultar Dados (DQL)

Escreva código Python para executar e imprimir os resultados das seguintes consultas:

1. Selecionar todos os professores e os nomes dos seus respectivos departamentos.
2. Encontrar todos os alunos inscritos numa cadeira específica (ex: 'Relational Databases').
3. Listar os cursos oferecidos por um departamento específico.
4. Atualizar o nome de um professor.
5. Apagar um curso e observar se a integridade referencial (foreign keys) é mantida.

Dica: Para ver os resultados, utilize `cursor.fetchall()`.

Exercício 5: Controlo de Transações (TCL)

Demonstre a propriedade "Tudo ou Nada":

1. Inicie uma transação.
 2. Tente inserir um aluno e uma inscrição.
 3. Provoque propositadamente um erro (ex: inserir numa tabela inexistente).
 4. Utilize `conn.rollback()` no bloco except e verifique que o aluno NÃO foi adicionado.
-

Exercício 6: Desafio de Normalização

Considere a seguinte tabela não normalizada `raw_data`:

StudentName	Course	Instructor	InstructorOffice	Grade
Alice	Databases	Dr. Smith	Room 101	A
Alice	Physics	Dr. Brown	Room 202	B
Bob	Databases	Dr. Smith	Room 101	C

1. Identifique as redundâncias e potenciais anomalias de atualização.
2. Decomponha esta tabela para a 3NF (Terceira Forma Normal) utilizando as tabelas que definimos no Exercício 2.
3. Escreva um script Python que leia dados de uma lista de tuplos (representando a tabela acima) e povoe as suas tabelas normalizadas.